

CITS5017 Deep Learning

Topic 3: Deep Computer Vision Using Convolutional Neural Networks

Associate Professor Du Huynh
(Unit Coordinator and Lecturer)

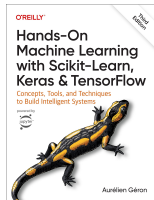
Semester 2, 2026



THE UNIVERSITY OF
WESTERN
AUSTRALIA

Chapters 14.

Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow



Deep Computer Vision Using Convolutional Neural Networks (CNNs)

Here are the main topics we will cover:

- Convolutional layers
- Filters and feature maps
- Stacking multiple feature maps
- Pooling layers
- TensorFlow implementation for CNNs
- Memory requirements of CNNs
- CNN architectures
- Using pretrained models and transfer learning

Convolutional Neural Networks (CNNs)

Convolutional neural networks (CNNs) emerged from the study^{1,2} of the brain's visual cortex.

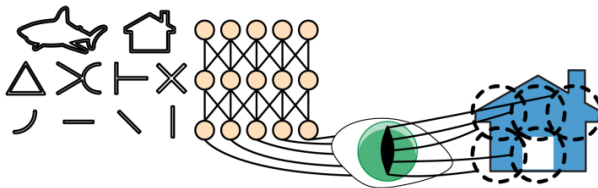


Figure 14-1. Biological neurons in the visual cortex respond to specific patterns in small regions of the visual field called *receptive fields*; as the visual signal makes its way through consecutive brain modules, neurons respond to more complex patterns in larger receptive fields.

¹David H. Hubel, "Single Unit Activity in Striate Cortex of Unrestrained Cats," *The Journal of Physiology* 147 (1959): 226-238.

²David H. Hubel and Torsten N. Wiesel, "Receptive Fields of Single Neurons in the Cat's Striate Cortex," *The Journal of Physiology* 148 (1959): 574-591.

Convolutional Neural Networks (CNNs)

These studies of the visual cortex inspired the *neocognition*³ (introduced in 1980), which gradually evolved into what we now call *convolutional neural networks*.

In the last few years, thanks to the increase in computational power, the amount of available training data, and the tricks presented in Chapter 11 for training deep nets, CNNs have managed to achieve superhuman performance on some complex visual tasks.

CNNs are not restricted to *visual perception*: they are also successful at other tasks, such as *voice recognition* or *natural language processing* (NLP).

³Kunihiko Fukushima, "Neocognitron: A Self-Organizing Neural Network Model for a Mechanism of Pattern Recognition Unaffected by Shift in Position," *Biological Cybernetics* 36 (1980): 193-202.

Convolution: A Brief Introduction

Given an image $\mathcal{I}$ and a 3×3 filter f , the convolution output of the input image $\mathcal{I}$ by f at pixel (x, y) is given by

$$\mathcal{I}'(x, y) = \sum_{i=-1}^1 \sum_{j=-1}^1 \mathcal{I}(x+i, y+j) f(i, j),$$

where $\mathcal{I}'$ is the output image.

It is easier to understand the convolution operation through an animation, e.g.

<https://www.youtube.com/watch?v=ulKbLD6BRJA> (1D convolution)

https://commons.wikimedia.org/wiki/File:2D_Convolution_Animation.gif

Some common 3×3 filters used in Computer Vision:

-1	0	1
-2	0	2
-1	0	1

-1	-2	-1
0	0	0
1	2	1

$\frac{1}{9}$

1	1	1
1	1	1
1	1	1

0	1	0
1	-4	1
0	1	0

(From left to right: **Sobel- x** (for detecting vertical edges); **Sobel- y** (for detecting horizontal edges); **uniform averaging filter** (for smoothing); **Laplacian operator** (2nd derivative filter).

Larger filters can also be defined. Odd size filters (5×5 , 7×7 , $\dots$) are usually preferred.

Convolutional Layers

The most important building block of a CNN is the *convolutional layer*.

- Neurons in the first convolutional layer are not connected to every single pixel in the input image, but only to pixels in their *receptive fields*.
- In turn, each neuron in the second convolutional layer is connected only to neurons located within a small rectangle in the first layer.

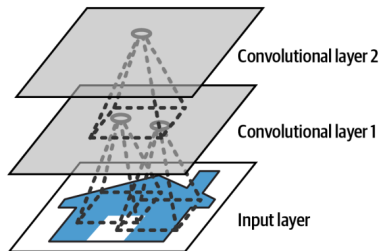
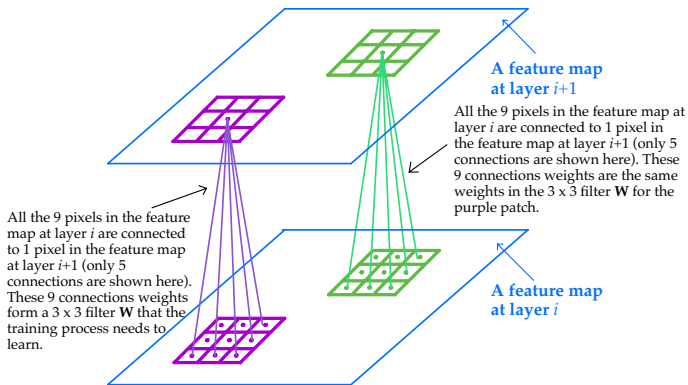


Figure 14-2. CNN layers with rectangular local receptive fields.

This architecture allows the network to concentrate on low-level features in the first hidden layer, then assemble them into higher-level features in the next hidden layer, and so on.

Convolutional Layers (cont.)



If only one 3×3 filter $\mathbf{W}$ is used for the convolution in layer i , then only 9 parameters from $\mathbf{W}$ plus a bias parameter need to be estimated, regardless of the numbers of rows and columns of the feature map in layer i .

Connections Between Layers

It is also possible to connect a large input layer to a much smaller layer by spacing out the receptive fields. The distance between two consecutive receptive fields is called the *stride*.

In the diagram, a 5×7 input layer (plus zero padding) is connected to a 3×4 layer, using 3×3 receptive fields and a stride of 2 in both directions, but it does not have to be so. A neuron located in row i , column j in the upper layer is connected to the outputs of the neurons in the previous layer located in rows $i s_h$ to $i s_h + f_h - 1$, columns $j s_w$ to $j s_w + f_w - 1$, where s_h and s_w are the vertical and horizontal strides.

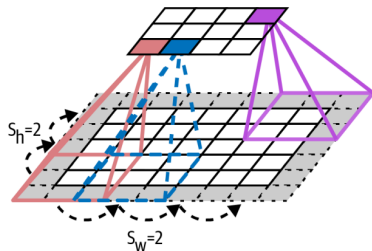


Figure 14-4. Reducing dimensionality using a stride of 2 in both directions ($s_w = s_h = 2$).

Filters

A **neuron's weights** can be represented as a **small image** having the same size as the receptive field. For example, consider the two **filters** (or **convolution kernels**) shown below:

- The first one is represented as a black square with a vertical white line in the middle (it is a 7×7 matrix full of 0s except for the central column, which is full of 1s); neurons using these weights will ignore everything in their receptive field except for the central vertical line (since all inputs will get multiplied by 0, except for the ones located in the central vertical line).
- The second filter is a black square with a horizontal white line in the middle. Once again, neurons using these weights will ignore everything in their receptive field except for the central horizontal line.



Filters (cont.)

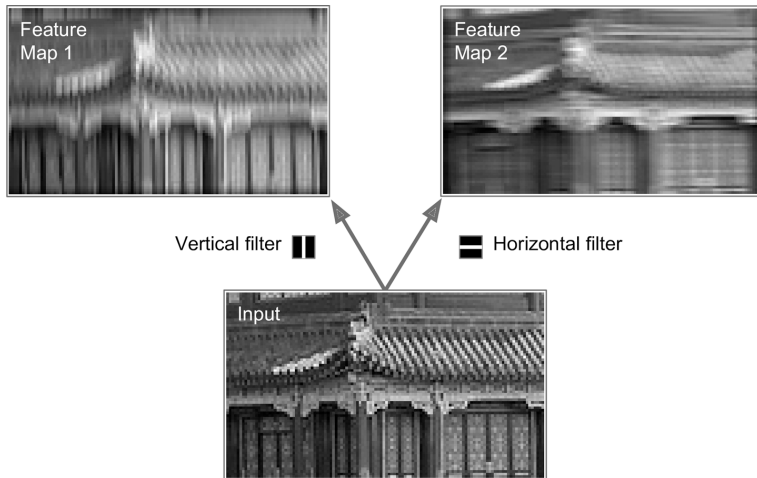


Figure 14-5. Applying two different filters to get two feature maps.

Stacking Multiple Feature Maps

So far, we have represented each convolutional layer as a **thin 2D layer**, but in reality it is composed of **several feature maps** stacked together, so it is more accurately represented in **3D**.

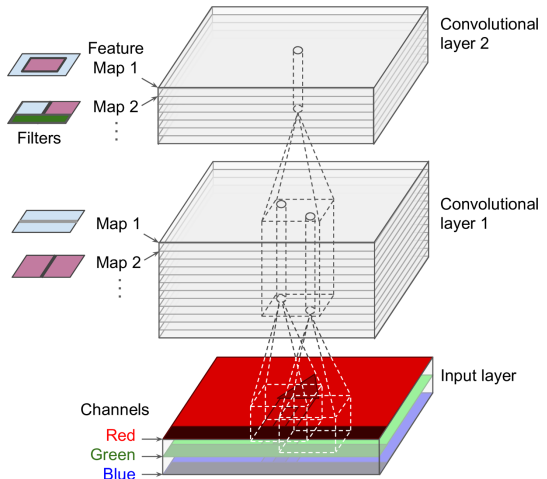


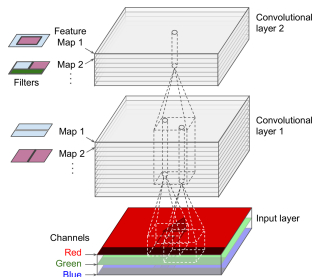
Figure 14-6. Convolution layers with multiple feature maps, and images with three channels

Stacking Multiple Feature Maps (cont.)

- Within one **feature map**, all neurons share the same parameters (the *weights* and the *bias* term of the filter). Neurons in different feature maps use different parameters. In fact, each feature map in a specific layer is generated by **convolving** a filter with the input of that layer.
- A neuron's receptive field is the same as described earlier, but it extends across all the previous layers' feature maps. In short, a convolutional layer simultaneously applies multiple filters to its inputs, making it capable of detecting multiple features anywhere in its inputs.
- Specifically, if a convolutional layer has n filters, then there are n feature maps generated for that layer. i.e., the output of that layer has n **channels**.

Stacking Multiple Feature Maps (cont.)

- A neuron located in row i , column j of the feature map k in a given convolutional layer ℓ is connected to the outputs of the neurons in the previous layer $\ell - 1$, located in rows $i s_h$ to $i s_h + f_h - 1$ and columns $j s_w$ to $j s_w + f_w - 1$, across all feature maps (in layer $\ell - 1$).
- Note that all neurons located in the same row i and column j but in different feature maps are connected to the outputs of the exact same neurons in the previous layer.



Stacking Multiple Feature Maps (cont.)

How the convolution is done...

Suppose that the dimension of the input image $\mathbf{x}$ is $N_{\text{height}} \times N_{\text{width}} \times N_{\text{channels}}$ and that there are K $n_h \times n_w$ filters altogether. Each filter f_k (for $k = 1, \dots, K$) must have N_{channels} channels in order for the convolution to be legal. That is, the dimension of each f_k is $n_h \times n_w \times N_{\text{channels}}$. These filters stack together to form a $n_h \times n_w \times N_{\text{channels}} \times K$ tensor.

The k^{th} output channel, denoted by z_k , is produced as follows:

$$z_k = b_k + \sum_{c=0}^{N_{\text{channels}}-1} \mathbf{x}_c * f_{c,k}$$

where $*$ denotes the convolution operation;

- $\mathbf{x}_c$ (whose dimension is $N_{\text{height}} \times N_{\text{width}}$) denotes channel c of $\mathbf{x}$;
- $f_{c,k}$ (an $n_h \times n_w$ matrix) denotes channel c of the filter f_k ;
- b_k is the bias term for the filter f_k .

So each filter produces one output channel (or one output feature map).

Implementing Convolutional Layers with Keras

In Keras,

- Each **input image** is typically represented as a **3D tensor** of shape `[height, width, channels]`.
- A **mini-batch** is represented as a **4D tensor** of shape `[minibatch_size, height, width, channels]`.
- The **weights of a convolutional layer** (i.e., all the **filters**) are represented as a 4D tensor of shape `[ $f_h$ ,  $f_w$ ,  $f_c$ ,  $f_n$ ]`, where f_h and f_w are the height of width of the filters, f_c is the number of channels (must be the same as the input image) of the filters, f_n is the total number of filters.
- The **bias terms**⁴ of a convolutional layer are simply represented as a **1D tensor** of shape `[ $f_n$ ]`.

⁴One bias term per filter in the convolution layer.

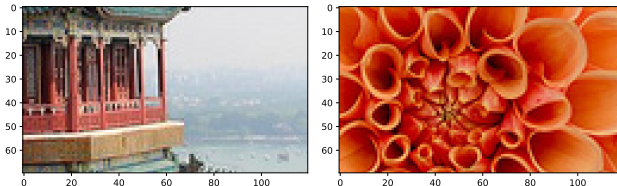
Implementing Convolutional Layers with Keras (cont.)

```
from sklearn.datasets import load_sample_images
import tensorflow as tf
import matplotlib.pyplot as plt

images = load_sample_images()["images"]
images = tf.keras.layers.CenterCrop(height=70, width=120)(images)
images = tf.keras.layers.Rescaling(scale=1 / 255)(images)

images.shape      # output: TensorShape([2, 70, 120, 3])

_, axes = plt.subplots(1,2, figsize=(10,4))
axes[0].imshow(images[0])
axes[1].imshow(images[1])
```

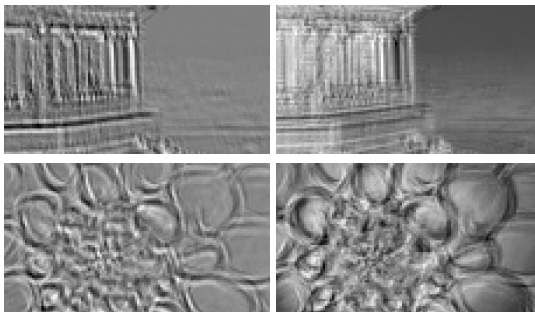


Implementing Convolutional Layers with Keras (cont.)

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7)
fmaps = conv_layer(images)

fmaps.shape    # output: TensorShape([2, 64, 114, 32])

_, axes = plt.subplots(2,2, figsize=(15,9))
for image_idx in (0,1):
    for fmap_idx in (0,1):
        axes[image_idx,fmap_idx].imshow(fmaps[image_idx, :, :, fmap_idx], cmap="gray")
        axes[image_idx,fmap_idx].axis("off")
```



Implementing Convolutional Layers with Keras (cont.)

Now let's look at the layer's weights. Just like a **Dense** layer, a **Conv2D** layer holds all the layer's weights, including the kernels and biases. The kernels are initialized randomly, while the biases are initialized to zero.

```
>>> kernels, biases = conv_layer.get_weights()
>>> kernels.shape
(7, 7, 3, 32)
>>> biases.shape
(32,)
```

The **kernels** array is 4D, and its shape is **[kernel_height, kernel_width, input_channels, output_channels]**. The **biases** array is 1D, with shape **[output_channels]**.

Notice that the shape of the input image is **[2, 70, 120, 3]** but the shape of the output feature maps, **fmaps**, is **[2, 64, 114, 3]**, i.e., the spatial dimension of the output is smaller.

Padding Options

The default padding option is “**valid**”, which means **zero-padding** (the convolution is *valid* in the sense that it does not go outside the image boundary). If we set **padding="same"** in our example on the previous slide, the output feature maps will have the shape **[2, 70, 120, 32]**.

```
conv_layer = tf.keras.layers.Conv2D(filters=32, kernel_size=7,  
                                     padding="same")  
  
fmaps = conv_layer(images)  
fmaps.shape          # output: TensorShape([2, 70, 120, 32])
```

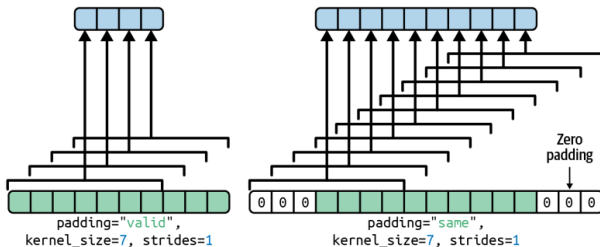


Figure 14-7. Padding options. Left diagram (**padding="valid"**): the output has 6 fewer pixels than the input. Right diagram (**padding="same"**): the output has the same number of pixels as the input.

Padding Options (cont.)

If the stride is greater than 1 (in any direction), then the output size will not be equal to the input size, even if `padding="same"`.

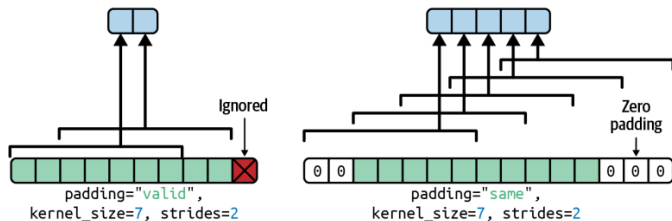


Figure 14-8. With strides greater than 1, the output is much smaller even when using "same" padding (and "valid" padding may ignore some inputs).

Memory Requirements of CNNs

A problem with CNNs is that the convolutional layers require a huge amount of RAM, especially during training, because the reverse pass of backpropagation requires all the intermediate values computed during the forward pass.

For example, consider a convolutional layer having 200 5×5 filters with stride 1 and “same” padding. If the input is a 150×100 RGB image. **Question: How many parameters need to be trained?**

The answer is: $(5 \times 5 \times 3 + 1) \times 200 = 15,200$.

(Note that the no. of parameters is independent of the height and width of the input image.)

Compared to a dense (fully connected) layer⁵, the above number is actually very small.

⁵ A fully connected layer with 150×100 neurons, each connected to all $150 \times 100 \times 3$ inputs, would have $150^2 \times 100^2 \times 3 = 675$ million parameters!

Memory Requirements of CNNs (cont.)

Each of the 200 feature maps contains 150×100 neurons, and each of these neurons needs to compute a weighted sum of its $5 \times 5 \times 3 = 75$ inputs: that's a total of $15,000 \times 75 \times 200 = 225$ million float multiplications. Not as bad as a fully connected layer, but still quite computationally intensive. This is for one training instance!

If we use single precision floats, i.e., 4 bytes (or 32 bits) per number, to store the feature maps, the memory requirement is: $150 \times 100 \times 200 \times 4 = 12$ MB of RAM. This is just for 1 instance.

If there are 100 instances in the minibatch, then it would use up **1.2 GB of RAM!**

Memory Requirements of CNNs (cont.)

During testing (making prediction), the RAM occupied by one layer can be released as soon as the next layer has been computed, so you only need as much RAM as required by two consecutive layers. However, during training, everything computed during the forward pass needs to be preserved for the reverse pass, so the amount of RAM needed is (at least) the total amount of RAM required by all layers.

If the training of your CNN crashes because of an out-of-memory error, you can try

- i reducing the mini-batch size;
- ii reducing dimensionality using a larger stride value;
- iii removing a few layers (making your CNN less deep);
- iv using 16-bit floats instead of 32-bit floats; and/or
- v distributing your CNN across multiple devices.

Pooling Layers

Pooling layers are the second common building block of CNNs.

- The pooling layers are quite easy to grasp. Their goal is to *subsample* (i.e., shrink) the input image in order to reduce the computational load, the memory usage, and the number of parameters (thereby limiting the risk of overfitting).
- Just like in convolutional layers, each neuron in a pooling layer is connected to the outputs of a limited number of neurons in the previous layer, located within a small rectangular receptive field.
- You must define the pooling layer's *size*, *stride*, and the *padding type*, just like before. However, a pooling neuron has no weights, as all it does is aggregate the inputs using an *aggregation function* such as the *max* or *mean*.

Pooling Layers (cont.)

In this example, we use a 2×2 max pooling kernel, a stride of 2, and no padding. Note that only the maximum input value in each receptive field makes it to the next layer, while the other inputs are dropped.

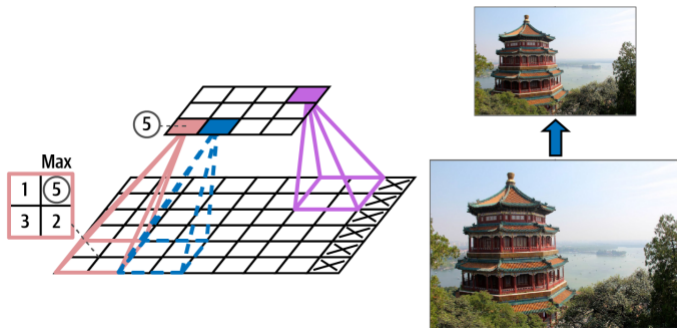


Figure 14-9. Max pooling layer (2×2 pooling kernel, stride 2, no padding).

Pooling Layers (cont.)

Max pooling vs average pooling

- Average pooling allows more information to be preserved, while max pooling preserves only the strongest features, getting rid of all the meaningless ones, so the next layer gets a cleaner signal to work with.
- Max pooling offers stronger *translation invariance* than average pooling, and it requires slightly less work to compute.

Average pooling layers used to be very popular, but people mostly use max pooling layers now, as they generally perform better. This may seem surprising, since computing the mean generally loses less information than computing the max.

Pooling Layers (cont.)

Implementing Pooling Layers with Keras

Implementing a **max pooling** layer or an **average pooling** layer in TensorFlow is quite easy – just use **MaxPooling2D** (alias **MaxPool2D**) for max pooling and **AveragePooling2D** (alias **AvgPool2D**) for average pooling. By default, the strides default to the kernel size and “valid” padding (i.e., no padding at all) is assumed. Use

Example 1: create a max pooling layer using a 2×2 kernel:

```
# both lines below are equivalent
max_pool = tf.keras.layers.MaxPooling2D(pool_size=2)
max_pool = tf.keras.layers.MaxPool2D(pool_size=2)
```

Example 2: create an average pooling layer using a 2×2 kernel:

```
# both lines below are equivalent
avg_pool = tf.keras.layers.AveragePooling2D(pool_size=2)
avg_pool = tf.keras.layers.AvgPool2D(pool_size=2)
```

Pooling Layers (cont.)

- A pooling layer typically works on every input channel independently, so the output depth is the same as the input depth.
- Unlike the convolutional layers where kernels have weights that need to be learned, kernels in pooling layers do not – they are just stateless sliding windows.
- Alternatively, we can pool over the depth dimension, in which case the image's spatial dimensions (height and width) remain unchanged, but the number of channels is reduced.

Note: You need to check the library whether pooling over the depth dimension is supported. e.g., Keras does not include a depthwise max pooling layer; however, it is not too difficult to implement the layer yourself.

Pooling Layers (cont.)

Although it is less common to perform max pooling along the depth dimension, **depthwise max pooling** allows the CNN to learn to be invariant to various features, e.g.,

- it could learn multiple filters, each detecting a different rotation of the same pattern (see Figure 14-11);
- it would ensure that the output is the same regardless of the rotation.

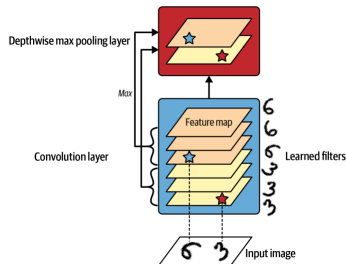


Figure 14-11. Depthwise max pooling can help the CNN learn to be invariant (to rotation in this case).

Pooling Layers (cont.)

Global Average Pooling

Similar to `AveragePooling2D`, *global average pooling* aggregates pixel values of each feature map in the spatial dimension; however it does that in a more global fashion – it averages all the pixel values in each feature map and reduces it to a single number.

Example:

```
...  
>>> images.shape  
TensorShape([2, 70, 120, 3])  
  
>>> global_avg_pool = tf.keras.layers.GlobalAvgPool2D()  
>>> global_avg_pool(images)  
<tf.Tensor: shape=(2, 3), dtype=float32, ...
```

Hyperparameters in a CNN

Unfortunately, convolutional layers in a CNN have quite a few hyperparameters: you must choose

- the number of filters,
- their height and width,
- the strides, and
- the padding type ("SAME" or "VALID").

Similarly, for the pooling layers, you also have the pool size, stride (default=`None`), and padding type (default=`"valid"`) to choose from.

As always, you can use cross-validation to find the right hyperparameter values, but this is very time-consuming. We will discuss common CNN architectures later, to give you some idea of what hyperparameter values work best in practice.

CNN Architectures

- Typical CNN architectures stack a few convolutional layers (each one generally followed by a ReLU layer), then a pooling layer, then another few convolutional layers (+ReLU), then another pooling layer, and so on.
- The image gets smaller and smaller as it progresses through the network, but it also typically gets deeper and deeper (i.e., with more feature maps).
- Near the output layer, a regular feedforward neural network is added, composed of a few fully connected layers (+ReLU), and the final layer outputs the prediction (e.g., a softmax layer that outputs estimated class probabilities).

CNN Architectures (cont.)

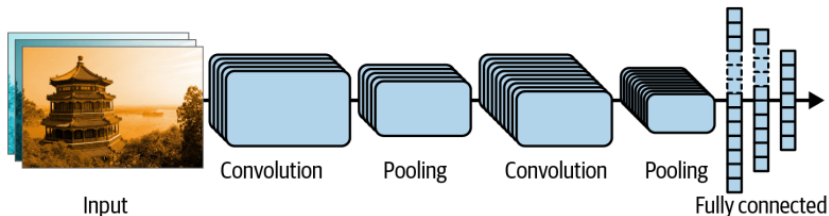


Figure 14-12. Typical CNN architecture

A common mistake is to use convolution kernels that are too large. It is better to stack two convolutional layers with 3×3 kernels instead of using a layer with a 5×5 kernel, as the network will use fewer parameters (also less computations) and will usually perform better.

One exception is for the first convolutional layer: it can typically have a large kernel (e.g., 5×5), usually with a stride of 2 or more: this will reduce the spatial dimension of the image without losing too much information.

CNN Architectures (cont.)

Example: A simple CNN to tackle the Fashion MNIST dataset.

```
from functools import partial
DefaultConv2D = partial(tf.keras.layers.Conv2D, kernel_size=3, padding="same",
                        activation="relu", kernel_initializer="he_normal")
model = tf.keras.Sequential([
    tf.keras.layers.InputLayer(shape=[28, 28, 1]),
    DefaultConv2D(filters=64, kernel_size=7),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=128),
    DefaultConv2D(filters=128),
    tf.keras.layers.MaxPool2D(),
    DefaultConv2D(filters=256),
    DefaultConv2D(filters=256),
    tf.keras.layers.MaxPool2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(units=128, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=64, activation="relu",
                          kernel_initializer="he_normal"),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(units=10, activation="softmax")
])
```

Compile the model using the **"sparse_categorical_crossentropy"** loss and train it using the training set. You should find that the CNN reaches over **92%** accuracy on the test set. It's not state of the art, but it is pretty good, and clearly much better than what we previously achieved with the MLP.

CNN Architectures (cont.)

Over the years, variants of this fundamental architecture have been developed, leading to amazing advances in the field. A good measure of this progress is the **error rate** in competitions such as the **ILSVRC⁶ ImageNet Challenge**. In this competition the **top-5 error rate⁷** for image classification fell from over **26%** to less than **3%** in just six years.

⁶ ILSVRC stands for *ImageNet Large Scale Visual Recognition Competition*. Also referred to as *ImageNet Challenge*

⁷ The top-five error rate is the number of test images for which the system's top five predictions did not include the correct answer.

CNN Architectures (cont.)

The LeNet-5 Architecture (1998)

The **LeNet-5** architecture is perhaps the most widely known CNN architecture. It was created by **Yann LeCun** in 1998 and widely used for handwritten digit recognition (MNIST). It is composed of the following layers:

Layer	Type	Maps	Size	Kernel size	Stride	Activation
Out	Fully connected	-	10	-	-	RBF
F6	Fully connected	-	84	-	-	tanh
C5	Convolution	120	1×1	5×5	1	tanh
S4	Avg pooling	16	5×5	2×2	2	tanh
C3	Convolution	16	10×10	5×5	1	tanh
S2	Avg pooling	6	14×14	2×2	2	tanh
C1	Convolution	6	28×28	5×5	1	tanh
In	Input	1	32×32	-	-	-

The main difference with more modern classification CNNs is the activation functions: today, we would use **ReLU** instead of **tanh** and **softmax** instead

of **RBF**. There were some other minor differences also.

CNN Architectures (cont.)

The AlexNet Architecture (2012)

- The AlexNet CNN architecture won the 2012 ImageNet ILSVRC challenge by a large margin: it achieved 17% top-5 error rate while the second best achieved only 26%!
- It was developed by Alex Krizhevsky (hence the name), Ilya Sutskever, and Geoffrey Hinton.
- It is quite similar to LeNet-5, only much larger and deeper, and it was the first to stack convolutional layers directly on top of each other, instead of stacking a pooling layer on top of each convolutional layer.

CNN Architectures (cont.)

The AlexNet Architecture (2012)

Layer	Type	Maps	Size	Kernel size	Stride	Padding	Activation
Out	Fully connected	—	1,000	—	—	—	Softmax
F10	Fully connected	—	4,096	—	—	—	ReLU
F9	Fully connected	—	4,096	—	—	—	ReLU
S8	Max pooling	256	6×6	3×3	2	valid	—
C6	Convolution	384	13×13	3×3	1	same	ReLU
C5	Convolution	384	13×13	3×3	1	same	ReLU
S4	Max pooling	256	13×13	3×3	2	valid	—
C3	Convolution	256	27×27	5×5	1	same	ReLU
S2	Max pooling	96	27×27	3×3	2	valid	—
C1	Convolution	96	55×55	11×11	4	valid	ReLU
In	Input	3(RGB)	227×227	—	—	—	—

To reduce overfitting, the authors used two **regularization techniques**:

- i **dropout** (with a 50% dropout rate) during training to the outputs of layers F9 and F10.
- ii **data augmentation** by randomly shifting the training images by various offsets, flipping them horizontally, and changing the lighting conditions.

CNN Architectures (cont.)

* **Data augmentation** artificially increases the size of the training set by generating many realistic variants of each training instance. As it helps reduce overfitting, it is considered as a regularization technique. For example, we can augment the training set by slightly shifting, rotating, and resizing every picture in the training set by various amounts and add the resultant pictures to the training set.

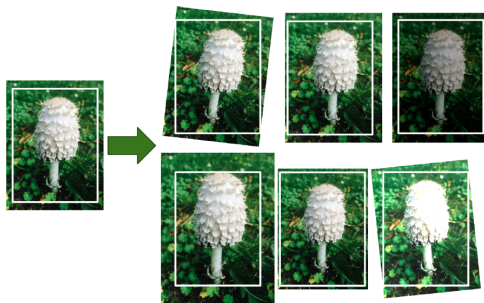


Figure 14-13. Generating new training instances from existing ones.

CNN Architectures (cont.)

The GoogLeNet Architecture (2014)

- The GoogLeNet architecture was developed by Christian Szegedy et al⁸ from Google Research.
- It won the ILSVRC 2014 challenge by pushing the top-5 error rate below 7%. This great performance came in large part from the fact that the network was **much deeper than previous CNNs** (see Figure 14-14). This was made possible by sub-networks called *inception modules*, which allow GoogLeNet to use parameters much more efficiently than previous architectures: GoogLeNet actually has 10 times fewer parameters than AlexNet (roughly 6 million instead of 60 million).

⁸Christian Szegedy et al., "Going Deeper with Convolutions," CVPR 2015.

CNN Architectures (cont.)

The GoogLeNet Architecture (2014)

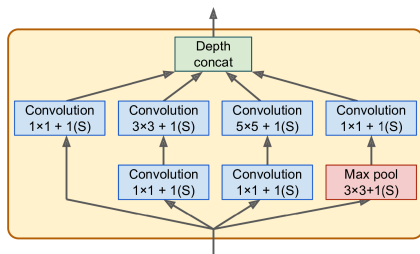


Figure 14-14. Inception module

The notation “ $3 \times 3 + 1(S)$ ” means that the layer uses a 3×3 kernel, stride 1, and “same” padding. All the conv. layers use the ReLU activation function.

The input signal is first copied and fed to four different layers.

The second set of convolutional layers use different kernel sizes (1×1 , 3×3 , and 5×5), allowing them to capture patterns at different scales.

The “same” padding used in all the layers ensure that the feature maps produced by them are of the same size. This allows the final step **depth concat** to concatenate along the depth dimension (**axis=3**).

CNN Architectures (cont.)

The GoogLeNet Architecture (2014)

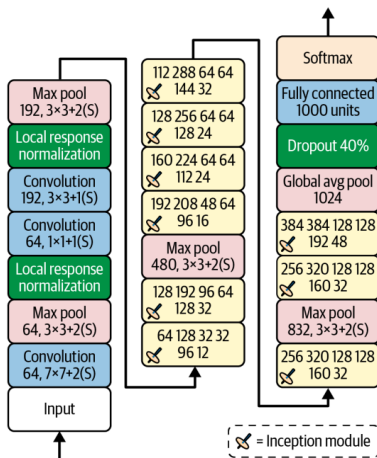


Figure 14-15. GoogLeNet architecture

Each input image in ImageNet is 256×256 ; there are 1000 classes; the network has 9 inception modules; the input to the first inception module is 32 (height) $\times 32$ (width) $\times 192$ (channels); the input that goes into the *Global avg pool* is $8 \times 8 \times 1024$.

For minibatches of size m , the data passed into the network will be a 4D tensor, e.g., the input that goes into the *Global avg pool* will be $m \times 8 \times 8 \times 1024$.

CNN Architectures (cont.)

The VGGNet Architecture (2014)

The runner-up in the ILSVRC 2014 challenge was VGGNet⁹ developed by Karen Simonyan and Andrew Zisserman from the Visual Geometry Group (VGG) research lab at Oxford University. It had a very simple and classical architecture, with 2 or 3 convolutional layers and a pooling layer, then again 2 or 3 convolutional layers and a pooling layer, and so on (reaching a total of just 16 or 19 convolutional layers, depending on the VGG variant), plus a final dense network with 2 hidden layers and the output layer. It used only 3×3 filters, but many filters.

⁹ Karen Simonyan and Andrew Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv preprint arXiv:1409.1556 (2014).

CNN Architectures (cont.)

The ResNet Architecture (2015)

The *Residual Network* (or just *ResNet*) proposed by Kaiming He et al¹⁰ won the ILSVRC Challenge in 2015 with an astounding top-five error rate under 3.6%. The winning variant used an extremely deep CNN composed of 152 layers (other variants have 34, 50, and 101 layers).

¹⁰ Kaiming He et al., "Deep Residual Learning for Image Recognition," arXiv preprint arXiv:1512:03385 (2015).

CNN Architectures (cont.)

The ResNet Architecture (2015)

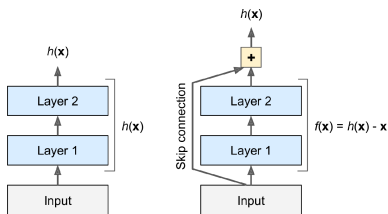


Figure 14-16. Residual learning

It confirmed the general trend: models are getting deeper and deeper, with fewer and fewer parameters. The key to being able to train such a deep network is to use *skip connections* (also called *shortcut connections*).

When training a neural network, the goal is to make it model a target function $h(\mathbf{x})$. By skipping connections and adding the feature $\mathbf{x}$ to a deeper layer, the network will be forced to model $f(\mathbf{x}) = h(\mathbf{x}) - \mathbf{x}$ rather than $h(\mathbf{x})$. This is called *residual learning*.

CNN Architectures (cont.)

More recent CNN architectures include:

Xception¹¹ (2016). This is a variant of GoogLeNet proposed in 2016 by Francois Chollet (the author of Keras), and it significantly outperformed Inception-v3 on a huge vision task (350 million images and 17,000 classes).

SENet¹² (2017). This is the winning architecture in the ILSVRC 2017 challenge. SENet stands for *Squeeze-and-Excitation Network*. This architecture extends existing architectures such as inception networks and ResNets, and boosts their performance. This allowed SENet to win the competition with an astonishing 2.25% top-five error rate!

Other noteworthy architectures include: *ResNeXt* (2016), *DenseNet* (2016), *MobileNet* (2017), *CSPNet* (2019), *EfficientNet* (2019).

¹¹Francois Chollet, "Xception: Deep Learning with Depthwise Separable Convolutions," arXiv preprint arXiv:1610.02357 (2016).

¹²Jie Hu et al., "Squeeze-and-Excitation Networks," CVPR 2018, pp. 7132-7141.

Using Pretrained Models from Keras

In general, you won't have to implement standard models like GoogLeNet or ResNet manually, since pretrained networks are readily available with a single line of code in the `tf.keras.applications` package. e.g.,

```
from sklearn.datasets import load_sample_images
model = tf.keras.applications.ResNet50(weights="imagenet")
images = np.array(load_sample_images()["images"], dtype=float32)
images_resized = tf.keras.layers.Resizing(height=224, width=224,
                                           crop_to_aspect_ratio=True)(images)
# images are converted from RGB to BGR; each color channel is zero-centered
inputs = tf.keras.applications.resnet50.preprocess_input(images_resized)

>>> Y_proba = model.predict(inputs)
>>> Y_proba.shape
(2, 1000)

top_K = tf.keras.applications.resnet50.decode_predictions(Y_proba, top=3)
for image_index in range(len(images)):
    print(f"Image #{image_index}")
    for class_id, name, y_proba in top_K[image_index]:
        print(f" {class_id} - {name:12s} {y_proba:.2%}")
```

Using Pretrained Models from Keras (cont.)

The output looks like this:

```
Image #0
n03877845 - palace      54.69%
n03781244 - monastery   24.72%
n02825657 - bell_cote   18.55%
Image #1
n04522168 - vase        32.66%
n11939491 - daisy       17.81%
n03530642 - honeycomb   12.06%
```

The correct classes are *palace* and *dahlia*. So, the model is correct for the first image but wrong for the second. However, that's because *dahlia* is not one of the 1,000 ImageNet classes.

What if you want to use an image classifier for classes of images that are not part of ImageNet? In that case, you may still benefit from the pretrained models by using them to perform *transfer learning*.

Pretrained Models for Transfer Learning

Let's train a model to classify pictures of flowers, reusing a pretrained **Xception** model.

```
import tensorflow as tf
import tensorflow_datasets as tfds

(test_set_raw, valid_set_raw, train_set_raw), info = tfds.load("tf_flowers",
    split=["train[:10%]", "train[10%:25%]", "train[25%:]"],
    as_supervised=True, with_info=True)
dataset_size = info.splits["train"].num_examples # 3670
class_names = info.features["label"].names # ["dandelion", "daisy", "tulips", "sunflowers", "roses"]
n_classes = info.features["label"].num_classes # 5

batch_size = 32
preprocess = tf.keras.Sequential([
    tf.keras.layers.Resizing(height=224, width=224, crop_to_aspect_ratio=True),
    # Xception requires the input pixel values in the range between -1 and 1
    tf.keras.layers.Lambda(tf.keras.applications.xception.preprocess_input)
])
train_set = train_set_raw.map(lambda X, y: (preprocess(X), y))
train_set = train_set.shuffle(1000, seed=42).batch(batch_size).prefetch(1)
valid_set = valid_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
test_set = test_set_raw.map(lambda X, y: (preprocess(X), y)).batch(batch_size)
```

Pretrained Models for Transfer Learning (cont.)

```
# load the Xception model
base_model =
tf.keras.applications.xception.Xception(weights="imagenet", include_top=False)

avg = tf.keras.layers.GlobalAveragePooling2D()(base_model.output)
output = tf.keras.layers.Dense(n_classes, activation="softmax")(avg)
model = tf.keras.Model(inputs=base_model.input, outputs=output)

# freeze the pretrained layers, compile and train for a few epochs
for layer in base_model.layers:
    layer.trainable = False

optimizer = tf.keras.optimizers.SGD(learning_rate=0.1, momentum=0.9)
model.compile(loss="sparse_categorical_crossentropy", optimizer=optimizer, metrics=["accuracy"])
history = model.fit(train_set, validation_data=valid_set, epochs=3)

# unfreeze some layers and train for a few more epochs
for layer in base_model.layers[56:]:
    layer.trainable = True

optimizer = tf.keras.optimizers.SGD(learning_rate=0.01, momentum=0.9)
model.compile(loss='sparse_categorical_crossentropy', optimizer=optimizer, metrics=['accuracy'])
history = model.fit(train_set, validation_data=valid_set, epochs=10)
```

Using this model, you should be able to get around 92% test accuracy. Training the model a bit longer should allow your test accuracy to reach 95% to 97%.

Summary

- Understand how the input layer and different convolutional layers are connected
- Understand the purpose of filters and what they generate (the feature maps)
- Know how to stack multiple feature maps
- Understand the role of pooling layers in a CNN
- Know how to implement a simply CNN using TensorFlow
- Understand the large memory requirements of CNNs
- Know some basic CNN architectures
- Know how to implement transfer learning with a pretrained model

For the next topic...

Read up to Chapters 15 on **Processing Sequences Using RNNs and CNNs**

And that's all for this lecture.

Have a good week.